Prepare a checklist for the preparation of cloud and DevOps tools. AWS, Azure, and GCP

Here is a comprehensive checklist for preparing Cloud and DevOps tools specifically for AWS, Azure, and GCP, including foundational skills, core services, DevOps tools, and CI/CD integrations.

Cloud and DevOps Preparation Checklist

1Foundational Skills (Common for AWS, Azure, GCP)

Category	Checklist
OS Basics	Linux/Ubuntu Commands, File System, Permissions
Networking	VPC/Virtual Network, Subnets, CIDR, NAT, Route Tables, DNS, Firewalls
Version Control	Git, GitHub, GitLab, Bitbucket Basics
Scripting	Bash, Shell Scripting, Python Basics
Containers	Docker, Docker Compose, Container Registries
IaC Basics	Terraform, AWS CloudFormation, Azure ARM, GCP Deployment Manager
Monitoring Basics	Logging, Metrics, Alerts, Dashboards

---

2 AWS Cloud Checklist

Category	Checklist
Compute	EC2, Lambda, Elastic Beanstalk, ECS, EKS
Storage	S3, EFS, FSx, Glacier
Database	RDS, DynamoDB, Aurora, ElastiCache
Networking	VPC, Subnets, Route53, Load Balancer, API Gateway, Direct Connect
IAM & Security	IAM Policies, Roles, KMS, Secrets Manager, Security Groups
CI/CD Tools	CodeCommit, CodeBuild, CodeDeploy, CodePipeline
Monitoring	CloudWatch, CloudTrail, AWS Config, X-Ray
Serverless	Lambda, EventBridge, Step Functions

3 Azure Cloud Checklist

Category	Checklist
Compute	Virtual Machines, App Services, Azure Functions, AKS, Container Apps
Storage	Blob Storage, File Storage, Queue, Table Storage
Database	Azure SQL, Cosmos DB, MySQL, PostgreSQL, Redis Cache
Networking	VNet, Load Balancer, Application Gateway, ExpressRoute, Front Door
IAM & Security	Azure AD, Role-Based Access Control (RBAC), Key Vault, NSGs
CI/CD Tools	Azure Repos, Azure Pipelines, Azure Artifacts, GitHub Actions
Monitoring	Azure Monitor, Log Analytics, Application Insights, Security Center
Serverless	Azure Functions, Logic Apps, Event Grid

4 GCP Cloud Checklist

Category	Checklist
Compute	Compute Engine, App Engine, Cloud Run, Cloud Functions, GKE
Storage	Cloud Storage, Filestore, Persistent Disk
Database	Cloud SQL, Firestore, Bigtable, Memorystore, Spanner
Networking	VPC, Load Balancer, Cloud CDN, Cloud Interconnect
IAM & Security	IAM Roles, Service Accounts, KMS, Secret Manager, Firewall Rules
CI/CD Tools	Cloud Source Repositories, Cloud Build, Cloud Deploy, Artifact Registry
Monitoring	Cloud Monitoring, Cloud Logging, Error Reporting, Cloud Trace, Cloud Debugger
Serverless	Cloud Functions, Cloud Run, Pub/Sub, Workflows

5 DevOps Tools Checklist (Cross-Cloud)

Category	Checklist
Source Code Management	GitHub, GitLab, Bitbucket
CI/CD Tools	Jenkins, GitHub Actions, GitLab CI, Azure Pipelines, AWS CodePipeline, GCP Cloud Build
Containerization	Docker, Podman
Container Orchestration	Kubernetes, AWS EKS, Azure AKS, GCP GKE
IaC (Infrastructure as Code)	Terraform, Pulumi, CloudFormation (AWS), ARM/Bicep (Azure), Deployment Manager (GCP)
Configuration Management	Ansible, Chef, Puppet
Monitoring & Logging	Prometheus, Grafana, ELK Stack (Elastic, Logstash, Kibana), Loki, Fluentd
Security & Compliance	HashiCorp Vault, SonarQube, Snyk, Trivy
Artifact Management	Nexus, JFrog Artifactory, GitHub Packages

---

6 CI/CD Pipeline Checklist

	Tasks
☐	Connect Source Code (GitHub, GitLab, Bitbucket)
☐	Build Automation (Maven, Gradle, npm, etc.)
☐	Testing Stage (Unit, Integration, Security Tests)
☐	Artifact Upload (S3, Azure Blob, GCP Storage, Artifactory)
☐	Deployment Automation (CodeDeploy, Azure Pipelines, GCP Cloud Deploy, Jenkins)
☐	Rollback Strategies
☐	Notification Integration (Slack, Teams, Email)

---

7 Additional Tools for DevOps Engineers

Category	Checklist
Secrets Management	AWS Secrets Manager, Azure Key Vault, GCP Secret Manager, HashiCorp Vault
Cost Management	AWS Cost Explorer, Azure Cost Management, GCP Billing Reports
Backup & DR	AWS Backup, Azure Recovery Services, GCP Backup Solutions
Incident Management	PagerDuty, OpsGenie, AWS Incident Manager, Azure Service Health

---

⭑ Soft Skills Checklist

	Checklist
☐	Documentation Writing (Markdown, Wiki)
☐	Communication with Dev/QA Teams
☐	Problem-Solving Mindset
☐	Agile / Scrum / Kanban Understanding

If you'd like, I can convert this checklist into a PDF, Excel, or Notion template for your easy tracking. Would you like that?

Prepare a checklist for the preparation of cloud and DevOps tools. Docker and Kubernetes

Here's a comprehensive checklist for Docker and Kubernetes preparation in the context of Cloud and DevOps roles:

Docker Preparation Checklist

☐ Basic Concepts

- Understand containerization vs virtualization
- Understand images, containers, and registries

- Docker architecture (Docker daemon, client, Docker Hub, registries)

 Installation & Setup

- Install Docker on Windows/Linux/Mac
- Configure Docker daemon and permissions

 Core Docker Commands

- docker run, docker build, docker pull, docker push
- docker ps, docker stop, docker start, docker rm
- docker exec, docker logs, docker inspect

 Dockerfile & Image Building

- Write Dockerfiles to create custom images
- Understand Docker image layers and caching
- Use .dockerignore to optimize builds

 Networking & Storage

- Docker networks: bridge, host, overlay
- Docker volumes and bind mounts for persistent storage

 Docker Compose

- Understand docker-compose.yml
- Create multi-container applications
- Commands: docker-compose up, down, logs, exec

 Security Basics

- Manage secrets securely (avoid hardcoding)
- Understand image scanning and vulnerabilities

 Docker Registry

- Push/pull images from DockerHub
- Set up private Docker registries

 Docker in CI/CD

- Dockerize applications for CI/CD pipelines
- Use Docker with Jenkins, GitHub Actions, GitLab CI, AWS CodeBuild

---

Kubernetes Preparation Checklist

☐ Basic Concepts

- Understand Kubernetes architecture (Master/Control Plane & Worker Nodes)
- Key components: API Server, etcd, Controller Manager, Scheduler, Kubelet, Kube-proxy
- Understand cluster, pods, nodes, namespaces

☐ Installation & Setup

- Install Minikube, Kind, or use cloud-managed Kubernetes (EKS, AKS, GKE)
- Install kubectl and configure kubeconfig

☐ Core Kubernetes Objects

- Pod
- ReplicaSet
- Deployment
- StatefulSet
- DaemonSet
- Job and CronJob
- Service (ClusterIP, NodePort, LoadBalancer)
- ConfigMap and Secret
- Namespace
- PersistentVolume (PV) and PersistentVolumeClaim (PVC)
- Ingress and Ingress Controllers

☐ Manifest Files (YAML)

- Understand structure: apiVersion, kind, metadata, spec
- Create manifests for all core objects
- Apply and delete manifests using kubectl apply -f and kubectl delete -f

☐ Networking & Load Balancing

- Kubernetes DNS
- Service discovery
- Ingress controllers (NGINX, AWS ALB Ingress, etc.)

☐ Monitoring & Logging

- Install and configure Prometheus, Grafana
- Fluentd, ElasticSearch, Kibana, or Loki for logging

☐ Security

- Role-Based Access Control (RBAC)
- Network Policies
- Pod Security Standards

- Secrets management

 Scaling & Auto Healing

- Horizontal Pod Autoscaler (HPA)
- Cluster Autoscaler
- Liveness and Readiness Probes

 Rolling Updates & Rollbacks

- Manage deployments with zero downtime
- Perform rollbacks using kubectl rollout undo

☒ Helm - Kubernetes Package Manager

- Install and use Helm charts
- Create custom Helm charts
- Manage releases with Helm

 CI/CD Integration

- Deploy apps to Kubernetes using Jenkins, GitHub Actions, GitLab CI, or ArgoCD
- Understand GitOps basics

 Bonus Cloud Native Tools to Explore

- ArgoCD / Flux (GitOps)
- Istio / Linkerd (Service Mesh)
- Kustomize (Configuration management)
- Velero (Backup/Restore)
- Lens (Kubernetes IDE)

If you'd like, I can create a detailed roadmap with hands-on exercises for Docker and Kubernetes. Would you like that?

Prepare a checklist for the preparation of cloud and DevOps tools. Jenkins, GitHub, GitLab

Here’s a comprehensive checklist for preparing Cloud and DevOps tools: Jenkins, GitHub, and GitLab. This checklist includes installation, configuration, integration, and usage essentials for interviews and real-world projects.

Jenkins Preparation Checklist

☐ Installation & Setup

- Install Jenkins (Local, AWS EC2, Docker, Kubernetes)
- Install required Java version (Java 11/17)
- Configure Jenkins with Admin user
- Configure firewall/security groups (Port 8080)

☐ Plugins Installation

- Git, GitHub, GitLab plugin
- Pipeline plugin
- Docker plugin
- Kubernetes plugin (if deploying on K8s)
- AWS, Azure, GCP integration plugins
- Slack/Email notification plugins

☐ Configuration

- Configure global tools (Git, JDK, Maven, Gradle, Docker)
- Configure credentials (SSH, API tokens, AWS keys)
- Setup environment variables

☐ Pipeline Setup

- Create freestyle jobs
- Create declarative/ scripted pipelines
- Write Jenkinsfile (Pipeline as Code)
- Connect Jenkins to GitHub/GitLab repositories
- Setup Webhooks for CI/CD

☐ Integration

- Integrate with GitHub/GitLab
- Docker (build, push, run containers)
- Kubernetes (deploy to K8s clusters)
- AWS, Azure, or GCP for deployment

☐ Security

- Configure Role-Based Access Control (RBAC)
- Secure credentials with Jenkins credentials store
- Backup Jenkins config/jobs

GitHub Preparation Checklist

☐ Setup

- Create a GitHub account
- Install Git on local machine
- Configure Git (git config user.name, git config user.email)
- Generate and add SSH key to GitHub

☐ Repository Management

- Create, clone, fork repositories
- Understand branching strategies (main, develop, feature, bugfix)
- Push, pull, merge, rebase, stash, cherry-pick

☐ Collaboration

- Create and manage Pull Requests (PR)
- Code review process
- Manage issues and discussions
- Use GitHub Projects and Actions

☐ CI/CD with GitHub Actions

- Write workflows (.github/workflows/main.yml)
- Build, Test, Deploy pipelines
- Integrate with AWS, Azure, GCP, Docker, Kubernetes

☐ Security & Management

- Configure branch protection rules
- Use Secrets for credentials in Actions
- Manage repository visibility (private/public)

GitLab Preparation Checklist

☐ Setup

- Create a GitLab account (cloud or self-hosted)
- Install Git on local machine
- Configure Git (git config user.name, git config user.email)
- Generate and add SSH key to GitLab

☐ Repository Management

- Create, clone, fork repositories

- Understand branching strategies
- Push, pull, merge, rebase

☐ CI/CD with GitLab

- Write .gitlab-ci.yml for pipeline configuration
- Define stages (build, test, deploy)
- Setup GitLab Runners (Shared, Group, or Specific)
- Integrate with AWS, Azure, GCP, Docker, Kubernetes

☐ Pipeline Features

- Use artifacts, cache, environments, and variables
- Setup triggers, scheduled pipelines, and manual jobs

☐ Security & Management

- Manage permissions (groups, roles)
- Configure Protected branches
- Use GitLab Secrets and Vault integration

Common Tools and Concepts Checklist (For Jenkins, GitHub, GitLab)

☐ Version Control

- Git basics (init, add, commit, push, pull, fetch, merge)
- Branching strategies (Git Flow, Trunk-based)
- Conflict resolution

☐ Docker Integration

- Write Dockerfile
- Build and push images
- Use Docker Compose

☒ Kubernetes Integration

- Understand YAML (Deployment, Service, ConfigMap, Secret)
- Deploy via Jenkins/GitHub/GitLab pipelines
- Use Helm charts

☁ Cloud Deployment

- Deploy to AWS, Azure, or GCP
- Manage cloud credentials securely

- Use cloud CLI tools (aws-cli, az-cli, gcloud)

□ Monitoring & Notifications

- Slack, Email, Teams notifications from pipelines
- Setup logs, metrics, and monitoring

□ Secrets Management

- Use Jenkins credentials store
- GitHub Secrets
- GitLab CI/CD variables

⭑ Interview Tips Checklist

- Explain Jenkins pipeline types (freestyle, declarative, scripted)
- How CI/CD works with GitHub and GitLab
- Real-time problem scenarios (e.g., rollback, failed deployment)
- How to secure credentials in pipelines
- Integration of Jenkins/GitHub/GitLab with AWS/GCP/Azure

If you'd like, I can convert this into a PDF checklist or a Notion template for your preparation. Want that? □

Prepare a checklist for the preparation of cloud and DevOps tools. Terraform, Chef, Puppet, SaltStack

Here's a detailed checklist for the preparation of Cloud and DevOps tools: Terraform, Chef, Puppet, and SaltStack. This checklist includes concepts, hands-on tasks, and best practices for each tool.

Terraform Checklist (Infrastructure as Code)

Category	**Items**
Basics	- Understand IaC concepts - Learn Terraform syntax (HCL)
Installation	- Install Terraform on local machine
Providers	- Configure AWS, Azure, GCP providers
Core Concepts	- Providers - Resources - Variables - Outputs - Data sources

State Management	- Understand terraform.tfstate - Remote backend (S3, Azure Blob, GCS)
Modules	- Create reusable modules - Use public modules from Terraform Registry
Commands	- init, plan, apply, destroy, validate, fmt, taint, import
Lifecycle Rules	- create_before_destroy, prevent_destroy
Provisioners	- Use local-exec and remote-exec
Workspaces	- Manage multiple environments (dev, staging, prod)
Versioning	- Lock provider versions - Use Terraform Cloud/Enterprise (optional)
Security	- Use Vault or AWS KMS for secrets - Secure remote state
Best Practices	- Code structure - Use .tfvars - DRY principles - Format code with fmt
Hands-on Projects	- Deploy VMs, VPC, S3, RDS, Load Balancers in AWS/Azure/GCP

Chef Checklist (Configuration Management)

Category	**Items**
Basics	- Understand Configuration Management concepts - Chef Architecture (Client-Server, Solo)
Installation	- Install ChefDK or Workstation - Install Chef Server and Client
Core Components	- Cookbooks - Recipes - Resources – Attributes
Knife Tool	- Configure Knife to interact with Chef Server
Cookbook Creation	- Generate cookbooks - Use knife cookbook create
Data Bags	- Store secrets and configurations securely
Environments	- Define dev, staging, production environments
Roles	- Create reusable roles for nodes
Testing	- Use Test Kitchen, ChefSpec, and Foodcritic
Automation	- Integrate Chef with CI/CD tools (Jenkins, GitHub Actions)
Security	- Manage secrets with Encrypted Data Bags - Chef Vault

Best Practices	- Write idempotent recipes - Use community cookbooks from Supermarket
Hands-on Projects	- Install Apache, MySQL, Docker, or setup a web server using Chef

---

Puppet Checklist (Configuration Management)

Category	**Items**
Basics	- Puppet Architecture (Master-Agent, Standalone) - Understand Manifest & Catalog
Installation	- Install Puppet Server and Puppet Agent
Core Components	- Manifests (.pp files) - Classes - Modules – Resources
Puppet Language	- Learn Puppet DSL (Domain Specific Language)
Modules	- Use Puppet Forge - Create custom modules
Hiera	- Externalize configuration with Hiera
Facter	- Use system facts in manifests
Environments	- Manage multiple environments
Automation	- Write idempotent code - Automate deployments
Testing	- Use rspec-puppet - Puppet Lint
Security	- SSL certificates for master-agent communication
Best Practices	- Parameterized classes - Use roles/profiles pattern
Hands-on Projects	- Configure web server, databases, user accounts, services using Puppet

---

SaltStack Checklist (Configuration Management + Automation + Remote Execution)

Category	**Items**
Basics	- Salt Architecture (Master, Minion, Syndic, Proxy)
Installation	- Install Salt Master and Minion
Core Concepts	- States - Pillars - Grains - Beacons – Reactors

Configuration Mgmt	- Write State files (SLS files)
Remote Execution	- Use Salt commands to execute tasks across nodes (salt '*' cmd.run 'uptime')
Pillars and Grains	- Manage variables and metadata
Orchestration	- Use Salt Orchestrate for complex deployments
Event-Driven Infra	- Configure Reactors and Beacons
Security	- Key management between master and minions
Automation	- Integrate with CI/CD pipelines
Best Practices	- Modular states - Use pillars for sensitive data - Version control for state files
Hands-on Projects	- Install and manage packages, services, users, firewall rules using SaltStack

---

☐ General DevOps Skills for All Tools

Skills	**Checklist**
Version Control	- Git, GitHub, GitLab
CI/CD Pipelines	- Jenkins, GitLab CI, GitHub Actions
Cloud Platforms	- AWS, Azure, GCP basics for provisioning
Scripting	- Bash, Python basics for automation
Networking Basics	- IP, DNS, Firewalls, Load Balancers
Monitoring/Logging	- Prometheus, Grafana, ELK stack basics
Security Practices	- Secret management, IAM, least privilege principles
Containerization	- Docker basics - Kubernetes (optional but highly recommended)
Infrastructure Design	- Understand high availability, scalability, fault tolerance

---

☆ Recommendation for Hands-on Practice:

- Deploy a 3-tier architecture:

- Provision with Terraform
- Configure OS and services using Chef, Puppet, or SaltStack
- Set up CI/CD pipelines to automate everything.

If you'd like, I can create a detailed roadmap, learning plan, or architecture diagram for any one (or combination) of these tools. Just ask! □

Prepare a checklist for the preparation of cloud and DevOps tools.AWS CloudFormation, Azure ARM, Google CDM

Here is a Checklist for the Preparation of Cloud and DevOps Tools: AWS CloudFormation, Azure ARM Templates, and Google Cloud Deployment Manager (CDM).

Checklist for AWS CloudFormation

	Topic	**Details**
□	Basics of AWS CloudFormation	Understand stacks, templates, change sets
□	Template Anatomy	- Resources- Parameters- Outputs- Mappings- Conditions- Metadata
□	Template Syntax	YAML and JSON formats
□	Intrinsic Functions	Fn::Join, Fn::Sub, Ref, Fn::GetAtt, Fn::ImportValue
□	Stack Operations	Create, Update, Delete, Rollback, Drift Detection
□	Nested Stacks	Modularize large templates
□	StackSets	Multi-account and multi-region deployments
□	Change Sets	Preview changes before applying
□	Security	IAM roles, permissions for CloudFormation
□	Best Practices	Reusability, modularity, parameters, outputs, rollback
□	CI/CD Integration	Integrate with CodePipeline, CodeBuild
□	Troubleshooting	Handle template errors, rollback failures

Checklist for Azure ARM Templates

	Topic	Details
☐	Basics of ARM Templates	Understand Resource Groups, deployments, templates
☐	Template Structure	Schema, Parameters, Variables, Resources, Outputs
☐	Template Language	JSON-based structure
☐	Functions and Expressions	concat, resourceId, reference, variables, parameters
☐	Linked Templates	Modular deployment using linked templates
☐	Deployment Modes	Incremental vs Complete
☐	Deployment Scopes	Management Group, Subscription, Resource Group
☐	Template Deployment Tools	Azure CLI, PowerShell, Azure Portal, REST API
☐	Role-Based Access Control (RBAC)	Permissions needed for deployment
☐	CI/CD Integration	Azure DevOps Pipelines, GitHub Actions
☐	Best Practices	Parameterization, modular templates, secure secrets
☐	Troubleshooting	Deployment errors, rollback, and error messages

---

Checklist for Google Cloud Deployment Manager (CDM)

	Topic	Details
☐	Basics of CDM	Understand deployments, templates, and configurations
☐	Template Format	YAML, Jinja2, Python
☐	Template Structure	Resources, Outputs, Imports
☐	Declarative vs Programmatic	Use Jinja2/Python for dynamic templates
☐	Resource Types	Supported GCP resources
☐	Deployment Commands	Create, update, delete deployments
☐	Parameterization	Use environment variables and input parameters

☐	IAM and Permissions	Roles needed for deployment
☐	CI/CD Integration	Cloud Build, GitHub Actions, Jenkins
☐	Best Practices	Modular templates, error handling, DRY principles
☐	Troubleshooting	View deployment logs, error messages, rollback

---

Common Across All Tools

	Topic	**Details**
☐	Infrastructure as Code (IaC)	Understand core IaC principles
☐	Version Control	Git, GitHub, GitLab, Bitbucket
☐	CI/CD Pipelines	Automate template deployment
☐	Secrets Management	AWS Secrets Manager, Azure Key Vault, Google Secret Manager
☐	Monitoring and Logging	CloudWatch, Azure Monitor, Stackdriver (GCP Monitoring)
☐	Cost Management	Understand billing impacts of IaC deployments
☐	Documentation	Document templates, parameters, and deployment steps

---

⭑ Learning Path Suggestion

1. Start with YAML/JSON fundamentals.
2. Learn basic template structures for each cloud.
3. Practice deploying small resources (e.g., VM, Storage).
4. Move to complex deployments with dependencies and parameters.
5. Learn to manage deployments via CLI, Portal, and CI/CD tools.
6. Handle errors and apply best practices.

If you'd like, I can also prepare sample templates for each—AWS CloudFormation, Azure ARM, and Google CDM. Would you like that? ☐

Prepare a checklist for the preparation of cloud and DevOps tools. Git

Here is a checklist for preparing and mastering Git, a fundamental tool for Cloud and DevOps engineers:

Git Preparation Checklist for Cloud & DevOps

□ Basic Setup & Installation

- Install Git (Windows, Linux, Mac)
- Configure Git with username and email
  git config --global user.name "Your Name"
  git config --global user.email "you@example.com"
- Set up SSH keys for authentication with remote repositories (GitHub, GitLab, etc.)
- Understand .gitignore files
- Understand basic Git configurations (git config --list)

□ Basic Git Commands (Local Repository)

- Initialize a repository (git init)
- Clone a repository (git clone)
- Check status (git status)
- Stage changes (git add)
- Commit changes (git commit -m "message")
- View commit history (git log / git log --oneline)
- Remove files (git rm)
- Move or rename files (git mv)

□ Branching & Merging

- Create branches (git branch)
- Switch branches (git checkout or git switch)
- Create and switch (git checkout -b or git switch -c)
- Merge branches (git merge)
- Understand and resolve merge conflicts
- Delete branches (git branch -d)

□ Remote Repository Management

- Add remote (git remote add origin <url>)
- Fetch from remote (git fetch)
- Pull updates (git pull)
- Push changes (git push)
- View remote repositories (git remote -v)

- Remove remote (git remote remove <name>)

 Tagging

- Create tags (git tag v1.0)
- Push tags (git push origin v1.0)
- Delete tags (git tag -d v1.0)

 Undo & Recovery

- Amend last commit (git commit --amend)
- Undo staged changes (git reset HEAD <file>)
- Discard changes in working directory (git checkout -- <file>)
- Rollback commits (git revert, git reset)
- Clean untracked files (git clean -f)

 Collaboration Workflow (Best Practices)

- Understand Git workflows (Git Flow, Feature Branch Workflow, Forking Workflow)
- Pull Requests (PR) / Merge Requests (MR) process
- Code Review Practices
- Writing meaningful commit messages

 Advanced Git Concepts

- Rebase (git rebase)
- Cherry-pick (git cherry-pick)
- Stash changes (git stash, git stash pop)
- Submodules (git submodule)
- Hooks (pre-commit, pre-push)
- Git Bisect (debugging with git bisect)
- Sparse checkout

 Git with CI/CD and DevOps

- Use Git in CI/CD tools (Jenkins, GitHub Actions, GitLab CI/CD, AWS CodePipeline)
- Branch strategies for deployments (main/master, develop, feature, release, hotfix)
- GitOps basics (managing infrastructure and Kubernetes manifests with Git)

□ Security & Best Practices

- Avoid committing sensitive data (use .gitignore, git-secrets)
- Scan for secrets (truffleHog, git-secrets, gitleaks)
- Sign commits with GPG (git commit -S)
- Backup repositories
- Protect branches with rules (e.g., no direct push to main)

□ Git Hosting Platforms (Hands-On)

- GitHub
- GitLab
- Bitbucket
- Azure Repos
- AWS CodeCommit

□ Tools & GUI Clients (Optional)

- GitKraken
- SourceTree
- GitHub Desktop
- VS Code Git Integration
- GitLens Extension (for VS Code)

□ Practice & Hands-On

- Create repositories (personal projects or dummy projects)
- Simulate team workflows (branching, PR, merge conflicts)
- Participate in open-source contributions

If you'd like, I can generate a PDF checklist version for download. Would you like that?

Prepare a checklist for the preparation of cloud and DevOps tools. Linux, Bash, Shell scripting

Here is a detailed Checklist for the Preparation of Cloud and DevOps Tools: Linux, Bash, and Shell Scripting.

Linux Essentials Checklist

Area	Topics	Status
Basics	Linux distributions (Ubuntu, CentOS, RHEL, etc.)	☐
	File system hierarchy (/, /home, /var, /etc)	☐
	Basic commands (ls, cd, pwd, mkdir, rm, cp, mv)	☐
	File permissions (chmod, chown, umask)	☐
	Process management (ps, top, kill, nice, htop)	☐
	Disk usage (df, du, mount, umount)	☐
	Network commands (ping, netstat, ss, curl, wget)	☐
	Package management (apt, yum, dnf, zypper)	☐
	Archive/Compression (tar, gzip, zip, unzip)	☐
	User and group management	☐
	Crontab and scheduled jobs	☐

Bash and Shell Scripting Checklist

Area	Topics	Status
Scripting Basics	Writing and executing simple scripts (#!/bin/bash)	☐
	File permissions for scripts (chmod +x)	☐
Variables	Defining, exporting, and using variables	☐
User Input	read command	☐
Conditional Logic	if, else, elif, case statements	☐
Loops	for, while, until loops	☐
Functions	Defining and calling functions	☐
Exit Status	$?, exit codes	☐

Operators	Arithmetic, logical, string comparison operators	☐
Input/Output	Redirecting (>, >>, <), piping (`	`)
File Testing	-e, -f, -d, -s etc. in conditions	☐
Error Handling	Trap errors, set -e, set -u	☐
Arrays	Declaring and using arrays	☐
String Manipulation	Extract, replace, length	☐
Logging	Create log files within scripts	☐
Debugging	Use set -x, set -e for debugging	☐
Interactive Scripts	Scripts that take arguments and provide output	☐
Automation	Combine Bash with Cron, SSH, SCP for automation	☐

Hands-On Practices Checklist

Task	**Status**
Create a user and automate user creation	☐
Write backup and restore scripts	☐
Monitor disk usage and send email alerts	☐
Automate package installation	☐
Write a script to start/stop services	☐
Automate log rotation and cleanup	☐
Automate SSH key generation and copying	☐
Create a script for EC2 instance health check	☐
Automate AWS CLI commands with Bash	☐

Cloud & DevOps Integration with Linux & Bash

Area	Topics	Status
Git + Bash	Automate git operations	☐
Docker + Bash	Write scripts to build, tag, push images	☐
Kubernetes + Bash (kubectl)	Automate pod management, deployments	☐
AWS CLI + Bash	Automate AWS tasks (EC2, S3, RDS)	☐
Terraform + Bash	Combine bash with Terraform commands	☐
CI/CD + Bash	Write pre/post build shell scripts for Jenkins/GitHub Actions	☐

Recommended Tools to Practice

- Terminal (Ubuntu WSL, EC2 Instance, VirtualBox, or VMware)
- Cloud Shell (AWS CloudShell, Azure Cloud Shell, Google Cloud Shell)
- VS Code + ShellCheck extension
- Git Bash (Windows)
- Online Linux Terminals (e.g., Katacoda, Webminal)

⭑ Tip:

If you want, I can also create a learning roadmap with tutorials, examples, and hands-on tasks. Just ask for it!